



2

Configuring the Development Environment and Creating the Project

To get started with WinUI and Windows App SDK development, it is important to install and configure Visual Studio for Windows desktop development. A WinUI developer must also understand the basics of application development with **Extensible Application Markup Language (XAML)** and C#, which we started learning in *[Chapter 1, Introduction to WinUI](#)*. However, the best way to understand the development concept is to get your hands on a real project. We will do that in this chapter.

After setting up your Visual Studio development environment, you will create the beginnings of a project that we will be building throughout the rest of the book.

In this chapter, you will learn the following topics:

- How to set up a new Visual Studio installation for Windows desktop application development
- How to create a new WinUI project, add a few controls, and run the project for the first time
- The anatomy of a new WinUI project and why each part is important

- How XAML can be used to build flexible, performant **user interfaces (UIs)**
- How WinUI fits with .NET and the role of each layer in the overall application architecture
- How to work with WinUI controls and customize them through changes in the XAML markup or C# code
- How to handle some basic UI events

If you are new to WinUI and other XAML-based development platforms, by the end of this chapter, you should be starting to feel comfortable working with WinUI projects.

Technical requirements

To follow along with the examples in this chapter, the following software is required:

- Windows 10, version 1809 (build 17763) or newer or Windows 11
- Visual Studio 2022 version 17.1 or later with the **.NET desktop development** workload and **Windows App SDK C# Templates** selected during installation

The source code for this chapter is available on GitHub at this URL:

<https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/main/Chapter02>.

Installing Visual Studio and Windows desktop development workloads

The first step to follow when starting with WinUI development is to install Microsoft's Visual Studio **integrated development environment (IDE)**. You can download the current version of Visual Studio 2022

from <https://visualstudio.microsoft.com/downloads/>. Visual Studio 2022 Community Edition is free for personal use and has all the features you will need to build WinUI applications.

TIP

If you want to try new Visual Studio features before they are released, you can install the latest Visual Studio Preview version from <https://visualstudio.microsoft.com/vs/preview/>. The Preview version is not recommended for the development of production applications as some features are unstable.

During installation, you can select workloads for any type of application that you want to create. For WinUI development, you must start by selecting the **.NET desktop development** workload. An overview of the **Workloads** section is shown in the following screenshot:

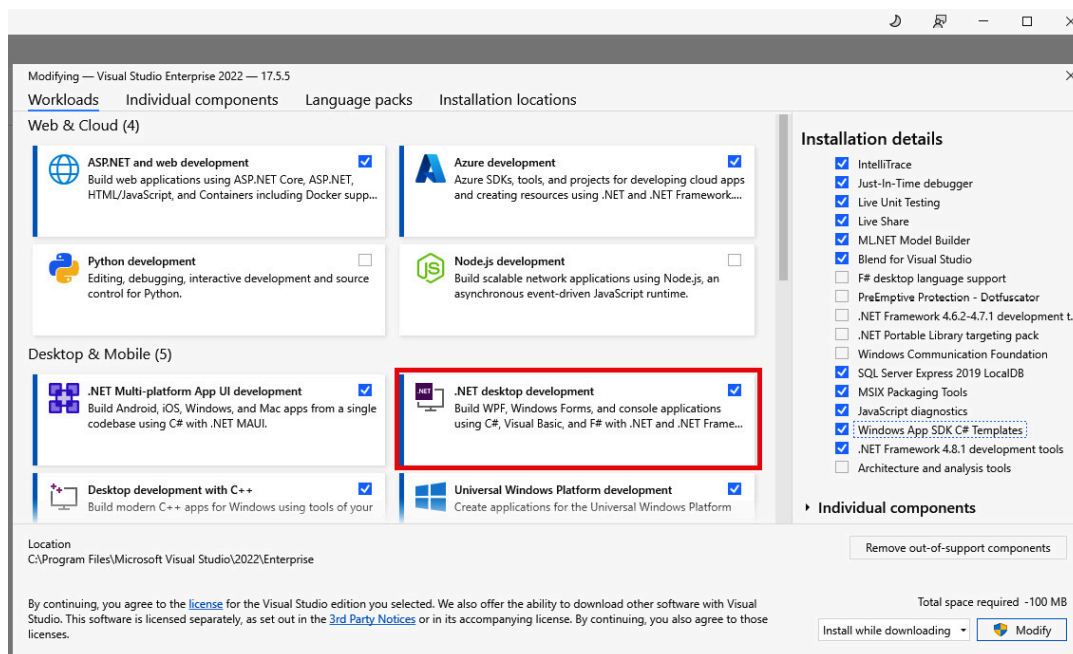


Figure 2.1 – Visual Studio Installer workload selection

After the **.NET desktop development** workload has been selected, select the **Windows App SDK C# Templates** component from the **Installation details** pane. These templates, required for building WinUI apps, are not installed by default with the workload.

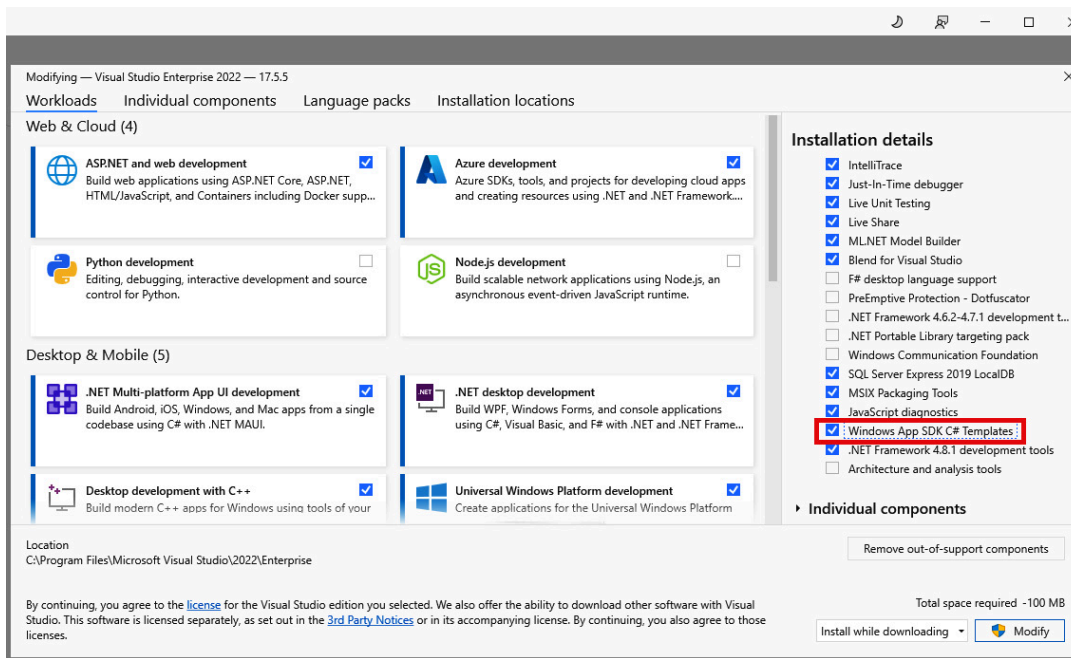


Figure 2.2 – Selecting the Windows App SDK C# Templates component for installation

NOTE

*If you are a C++ developer and would like to build WinUI apps with C++, you must select the **Desktop development with C++ workload** and the optional **Windows App SDK C++ Templates** component within the **Desktop development with C++ workload**. However, building WinUI applications with C++ is beyond the scope of this book.*

When continuing to the next step, the Visual Studio Installer will download and install all selected workloads and components. When setup is complete, launch Visual Studio. The first time you run Visual Studio, you will be prompted to sign in with a Microsoft account. Linking Visual Studio to your account will enable Visual Studio to sync your settings, link any available licenses, and link your Microsoft Store account after it has been created. We will discuss more about the Microsoft Store and application distribution in [Chapter 14, Packaging and Deploying WinUI Applications](#).

Now, it's time to start building our WinUI application.

Introducing the application idea

The application we're going to build is a tool called **My Media Collection**. It's a simple utility that can catalog your entire media library. Because digital media is becoming more popular as time goes on, we can design the application to support the inclusion of both physical and digital media. The application will be able to catalog different types of media, including music, video, and books. We'll add some features that will only light up for certain media types. Physical media (books, DVDs, and CDs) are often loaned to friends. This application will help you remember who borrowed your favorite book at a recent family party.

Reviewing the application features

Before we dive in and create the new project, let's get organized. It helps to understand what you're going to build so that you can track your progress while progressing through each chapter. If you're tracking your development on GitHub, you could create an **Issue** for each feature. Let's start with a high-level look at the application's features, as follows:

- View all media
- Filter the media library by the following: **Media Type (Music, Video, or Book)**, **Medium** (the available choices will vary by media type but will include **CD, Record, DVD, Blu-Ray, Hardcover, Paperback, Digital**), or **Location (In Collection or Loaned)**
- Add a new media item
- Edit a media item
- Mark an item as loaned or returned
- Application sign-in
- Back up (or restore) the collection data with OneDrive

The application will use features of the **Windows Community Toolkit**, which can simplify things such as Microsoft account authentication and file access on OneDrive. You can read more about the toolkit on Microsoft Learn:

<https://learn.microsoft.com/dotnet/communitytoolkit/windows/>.

The application's data will be stored in a local **SQLite** database, allowing for online or offline access to the media collection. You'll even be able to send email reminders if one of your friends is taking a little too long in returning one of the items in your collection.

WinUI in Desktop projects

In this chapter, we will be building a Windows App SDK application with the **WinUI in Desktop** project template. A **WinUI in Desktop** project targets the .NET runtime while using the same XAML schema as a UWP project.

A **WinUI in Desktop** project also includes a **Windows Application Packaging** project in the newly created solution. We will learn more about packaging WinUI applications in [Chapter 14](#), *Packaging and Deploying WinUI Applications*. Now, let's get started with our first project.

Creating your first WinUI project

It's time to start building the project. To do so, proceed as follows:

1. Launch Visual Studio, and from the opening screen, select **Create a new project**, as illustrated in the following screenshot:

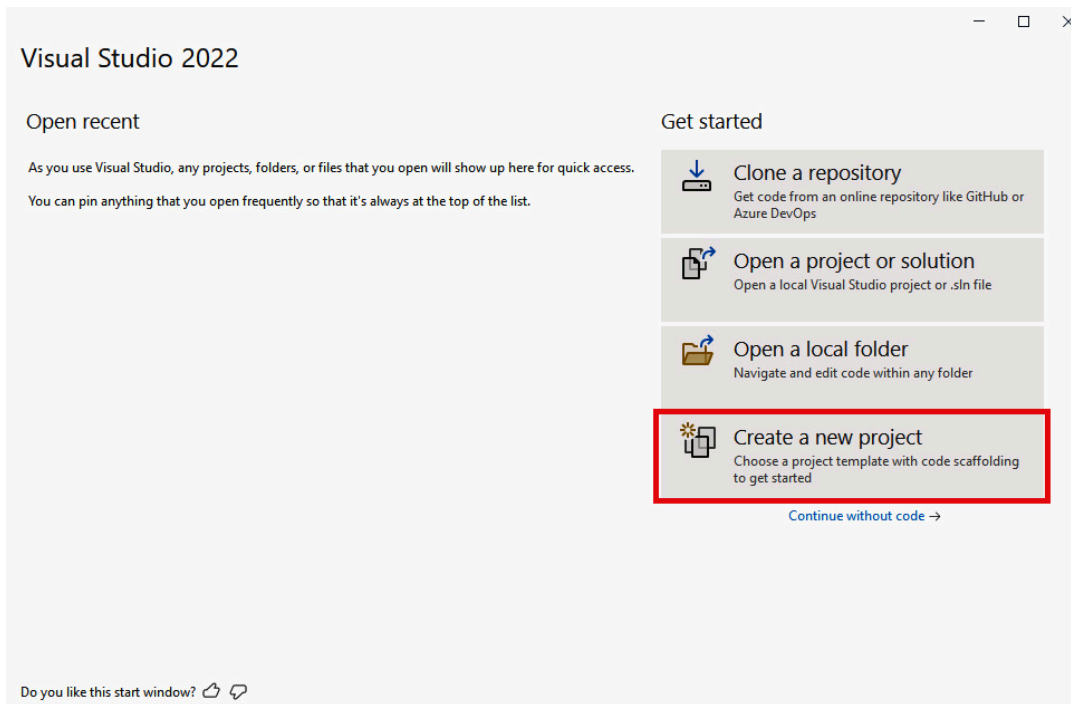


Figure 2.3 – The initial Visual Studio dialog

2. On the **Create a new project** screen, enter **winui** in the **Search for templates** field, select the **Blank App, Packaged (WinUI 3 in Desktop) C# template**, and click **Next**, as illustrated in the following screenshot:

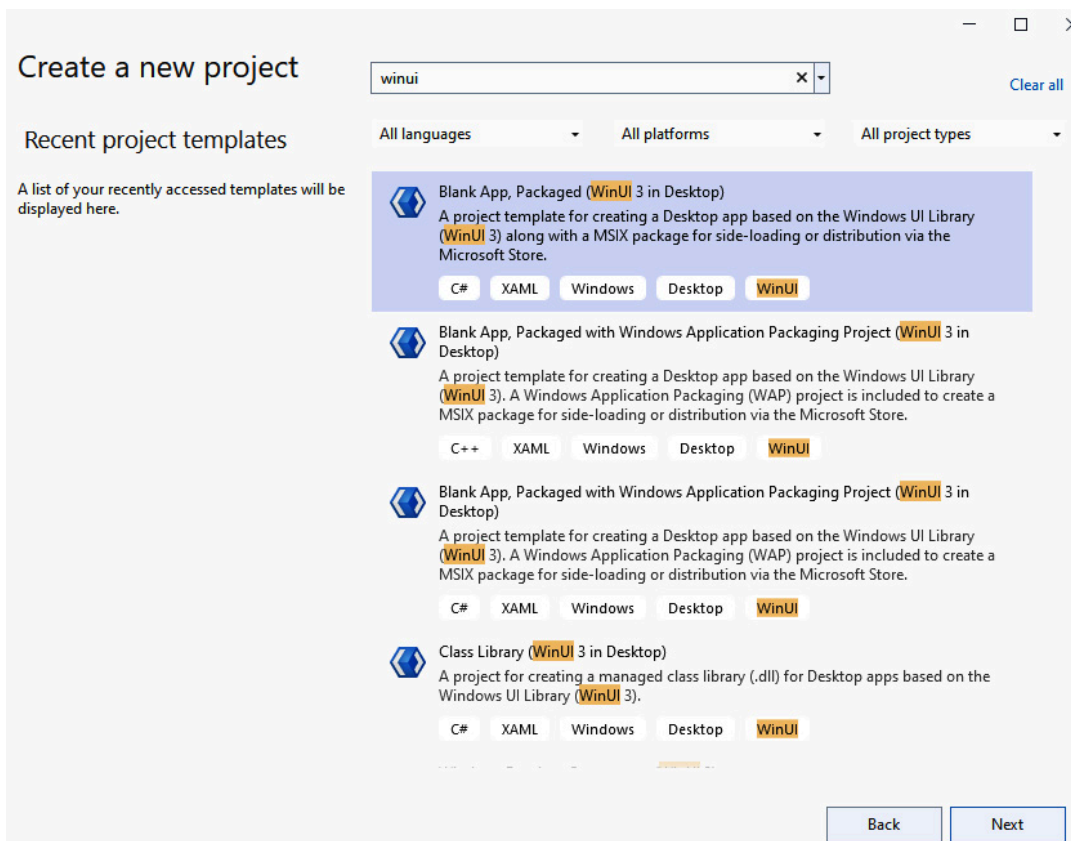


Figure 2.4 – Selecting the project template

TIP

*Be sure to select the **C#** project template and not **C++**. You can filter the project types to show only **C#** projects by changing the **Language** filter from **All languages** to **C#**.*

3. Name the project **MyMediaCollection**, leave the rest of the fields set to their default values, and click **Create**. You may be prompted to select the Windows versions to target with your app. You can leave these set to their default values for this project.

NOTE

*It is up to you whether you want to change these versions for your app, but you will be limiting the versions of Windows that can install your app. If you are using a control or feature that you know is only available in specific versions of Windows, you must select that version as the **Minimum version**. The **Target version** must be equal to or greater than the **Minimum version**. If you are unsure of what to choose, you should stick with the default values.*

4. Now, the project has been created and Visual Studio has loaded, it's always a best practice to build and run the project. Run the application and see what the template has provided for **MainWindow**. You should see an empty window hosting a few controls. Next, we will see what Visual Studio has created to get us started.

NOTE

*To run and debug WinUI apps on Windows, you must update your Windows settings to enable **Developer Mode**. To do this, follow these steps:*

1. Open **Settings** from the **Start** menu.

2. Type **Developer** in the search bar and select the **For Developers** settings from the search results.

3. On the **For developers** page that appears, switch the **Developer Mode** toggle switch on, if it is not already turned on. Enabling this allows developers to sideload, run, and debug unsigned apps, and enables some other developer-focused Windows settings. You can get more information here: <https://learn.microsoft.com/windows/apps/get-started/enable-your-device-for-development>.

Anatomy of a WinUI in Desktop project

Now that we have a new empty WinUI project loaded in Visual Studio, let's examine the different components. In **Solution Explorer**, you will see two XAML files, named **App.xaml** and **MainWindow.xaml**. We will start by discussing the purpose of each of these. Both files can be seen in the following screenshot of **Solution Explorer**:

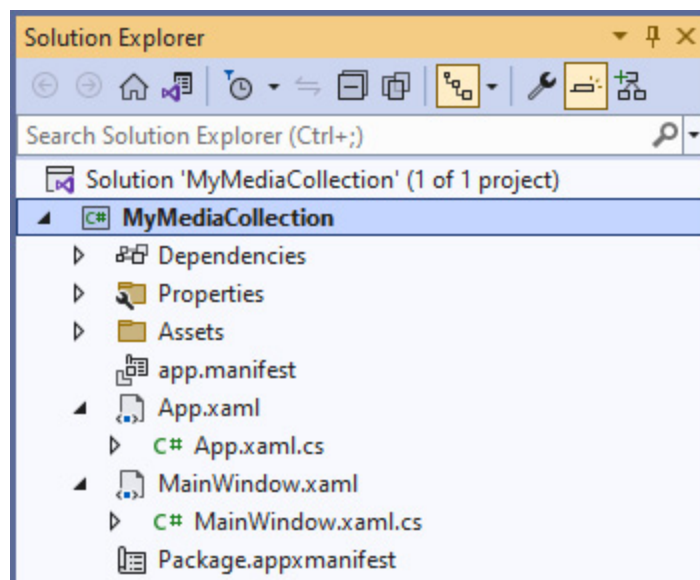


Figure 2.5 – The new WinUI solution in Solution Explorer

Reviewing App.xaml

The **App.xaml** file, as its name implies, stores resources available across the entire application. If you have any templates or styles that will need to be used across multiple windows, they should be added at the **Application** level.

The new project's **App.xaml** file will contain some initial markup, as illustrated in the following code snippet:

```
<Application
  x:Class="MyMediaCollection.App"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/
presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:local="using:MyMediaCollection">
  <Application.Resources>
    <ResourceDictionary>
      <ResourceDictionary.MergedDictionaries>
        <XamlControlsResources
          xmlns="using:Microsoft.UI
            .Xaml.Controls" />
        <!-- Other merged dictionaries here -->
      </ResourceDictionary.MergedDictionaries>
      <!-- Other app resources here -->
    </ResourceDictionary>
  </Application.Resources>
</Application>
```

We will cover some XAML basics in the next section. For now, you should know that the **Application.Resources** section will contain all the resources to be shared across an application. Within this section, the **ResourceDictionary.MergedDictionaries** section contains references to other XAML files whose resources will be shared with the entire application. This allows developers to organize shared content into multiple resource files, leading to better-organized and more maintainable XAML. This also enables the sharing of third-party resources across the application. For example, this file merges

XamlControlsResources from the **Microsoft.UI.Xaml.Controls** namespace. These are the resources for the WinUI controls that we will be using to build our app.

Reviewing App.xaml.cs

In **Solution Explorer**, expand the **App.xaml** file's node, and you will see there is another file named **App.xaml.cs** nested underneath. This is referred to as a **code-behind file**. It is a partial C# class that, in tandem with the XAML file, defines the **Application** class. If you open the C# file, you will see that it already contains some code. This is where you will handle any application-wide events. This is the event handler added by default: **OnLaunched**. If you need to execute any specific logic when your app is first launched, it should be added here. This is also where any application arguments passed to the app can be handled.

Reviewing MainWindow.xaml

The **MainWindow.xaml** file contains the **MainWindow** WinUI window that will be displayed when the application launches. You can see this in the **OnLaunched** event handler in **App.xaml.cs**, as illustrated in the following code snippet:

```
m_window = new MainWindow();  
m_window.Activate();
```

In a new blank WinUI app, **Window** will contain an empty **StackPanel** layout control. Try replacing **StackPanel**'s child controls with **TextBlock** with a **Text** property of **Media** and replace **StackPanel** with **Grid**. The result should look like this:

```
<Window  
    x:Class="MyMediaCollection.MainWindow"
```

```
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/
  presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="using:MyMediaCollection"
xmlns:d="http://schemas.microsoft.com/expression/blend/
  2008"
xmlns:mc="http://schemas.openxmlformats.org/
  markup-compatibility/2006"
mc:Ignorable="d">
<Grid>
    <TextBlock Text="Media"/>
</Grid>
</Page>
```

To prevent exceptions at runtime, remove the event handler for the **Button.Click** event in **MainWindow.xaml.cs**. Now run the app with the **Debug | Start Debugging** menu item or the **Start Debugging** button on the Visual Studio toolbar, and you should see something like this:



Figure 2.6 – MainWindow with TextBlock added

We'll make this window a bit more functional later in the chapter. For now, let's finish our review of the project structure.

Reviewing MainWindow.xaml.cs

The code-behind file for the **MainWindow.xaml.cs** window now contains only a call to **InitializeComponent()** in the constructor because we removed the event handler for the original **Button** control. Later, we will add some code here to populate some sample data and handle events on the page. In a well-designed MVVM app, the code-behind files for your pages will have very little code. Most of the code will reside in the **ViewModel** classes.

Reviewing the project references

All your project's references to **NuGet** packages, **dynamic link libraries (DLLs)**, and other projects in the solution will appear in the **Packages** folder under **Dependencies** in **Solution Explorer**. Your WinUI project will reference the following NuGet packages:

- **Microsoft.WindowsAppSDK**: The Windows App SDK components, which include WinUI
- **Microsoft.Windows.SDK.BuildTools**: The Windows SDK components required to build a WinUI solution

NOTE

Do not modify or remove any of these references.

Reviewing the project properties

If you right-click the project in **Solution Explorer** and select **Properties** from the context menu, you can view and modify the project properties. You won't often need to make any changes here. These are a couple of properties you might need to modify from time to time:

- **Assembly Name**: You can change the name of the output assembly that is compiled by the project
- **Min Version** and **Target Version**: The Windows versions that were selected when creating the project can be modified here

The primary difference between a new UWP project and a new WinUI 3 project is the controls and other objects that are referenced in the **Windows.UI.Xaml.*** namespaces are now referenced in the Windows App SDK's **Microsoft.UI.Xaml.*** namespaces. One other difference is that in the **App.xaml** file of the UWP project, it's not necessary to im-

port the controls' resources. The remaining differences are mostly hidden from app developers in the project file.

Now that you've become familiar with the WinUI project, let's start building the UI for **MainWindow**. We will start with some of the more common XAML controls and concepts.

XAML basics

It's time to start building the main screen of the **My Media Collection** application. The focal point of the application will be the media items in the collection. To display that list, we are going to need a few things, as follows:

- A **Model** class that defines an item in the collection
- Some code to bind the collection of items to the UI
- A XAML control to display the items

Building the model

We will start by building the model for the **My Media Collection** application. A **model** defines an entity and its attributes. Earlier in the chapter, we discussed some of the items' attributes we want to display in the UI. To display and (eventually) persist this information, we must create the model.

The initial version of our model will consist of two enumerations (**ItemType** and **LocationType**) in an **Enums** folder, and two classes (**Medium** and **MediaItem**) in a **Model** folder, as illustrated in the following screenshot:

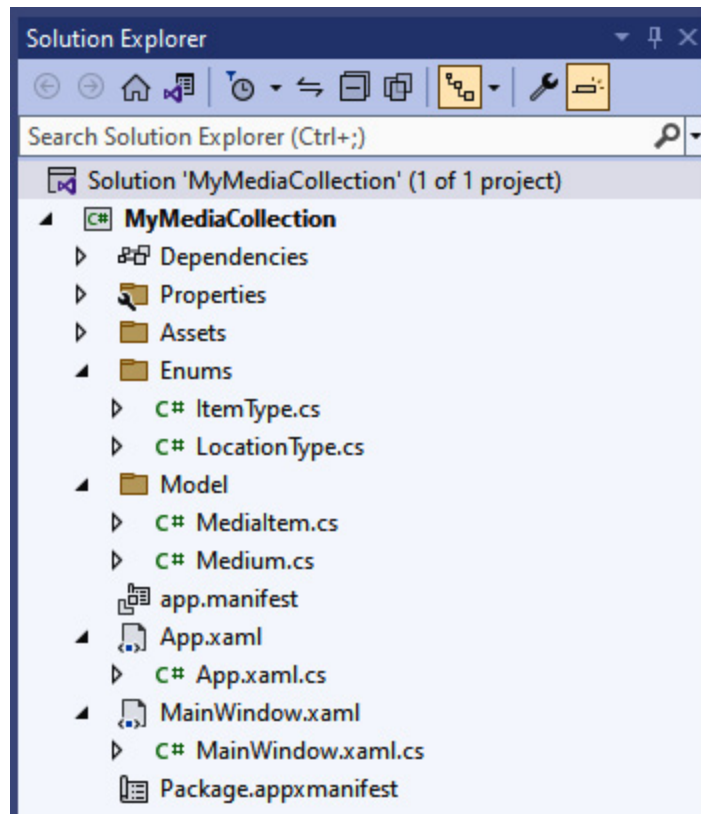


Figure 2.7 – Solution Explorer displays the new Model and Enum files

1. To add a new folder to the project, right-click on the **MyMediaCollection** project in **Solution Explorer** and select **Add | New Folder**. Enter **Enums** as the folder name and hit *Enter*. Repeat this process to add a **Model** folder.
2. Next, right-click the project file and select **Add | Class**. In the **Add New Item** dialog that appears, the **Class** file type will be selected. Because Visual Studio does not have a template for a new enum, keep this selection, change the name to **ItemType**, and click the **Add** button, as illustrated in the following screenshot:

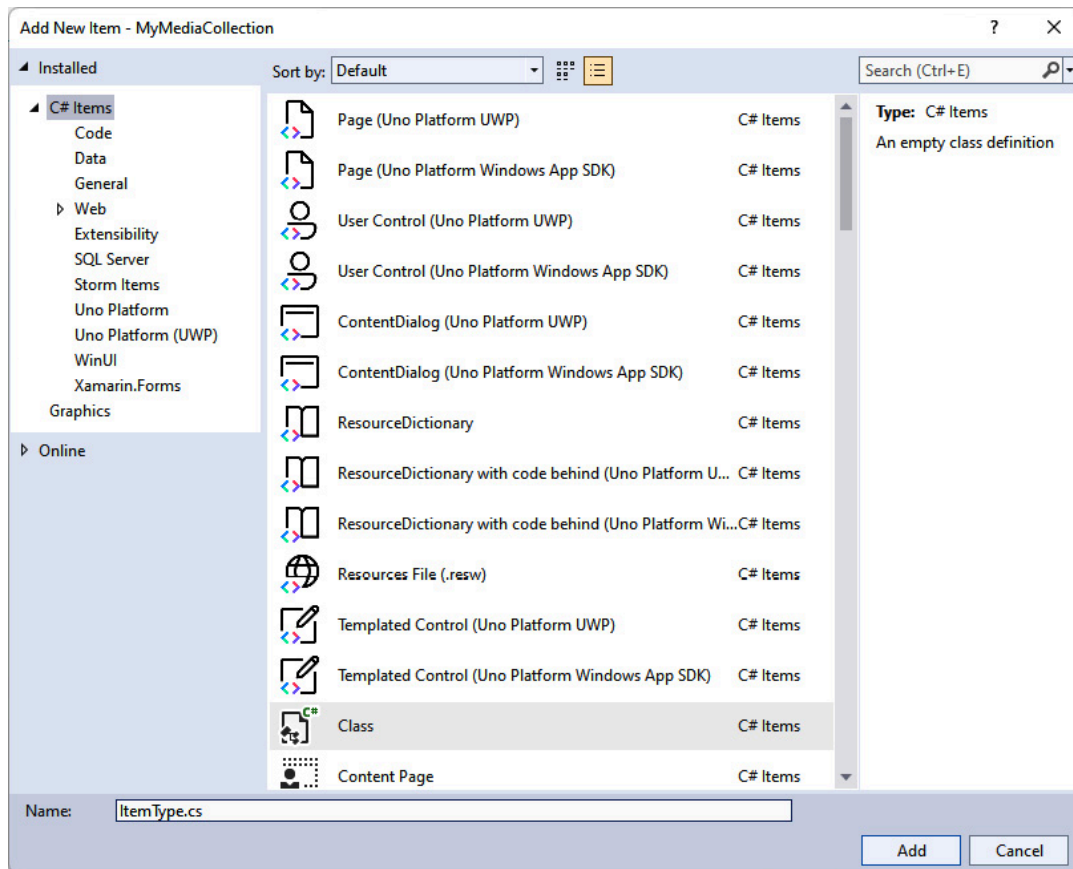


Figure 2.8 – The Add New Item dialog

3. The code for **ItemType** is created and displayed. Change the **class** definition to **enum** and change the **internal** keyword to **public**. The **ItemType** enum contains three possible values named **Music**, **Video**, and **Book**. When you're finished, the definition of the **ItemType** enum will look like this:

```
public enum ItemType
{
    Music,
    Video,
    Book
}
```

4. Repeat the steps to create a **LocationType** enum, defined as follows:

```
public enum LocationType
{
```



```
InCollection,  
    Loaned  
}
```

5. Using what you have learned, create two classes named **Medium** and **MediaItem** in the **Model** folder. The **Medium** class represents a specific medium such as *hardcover* or *paperback*, while the **MediaType** property assigns an **ItemType** to which the **Medium** class belongs. For these, the valid **ItemType** would be **Book**. When you are finished, the two new classes will look like this:

```
public class Medium  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public ItemType MediaType { get; set; }  
}  
public class MediaItem  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public ItemType MediaType { get; set; }  
    public Medium MediumInfo { get; set; }  
    public LocationType Location { get; set; }  
}
```

NOTE

*To reference the enum types in the **Enums** folder from the two classes, a **using** declaration will need to be added at the top of each new class, like this: **using MyMediaCollection.Enums;***

In addition to the properties discussed earlier in the chapter, an **Id** property has been added to the **MediaItem** class to uniquely identify each item in the collection. This will be useful later when we start persisting data.

Creating sample data

With the model classes in place, we're ready to add some code that will create three media items to display in the UI. This will help us visualize things as we start creating our list of items on the main screen. When this step is complete and we're ready to move on to enabling the ability to add items through the app, this code will be removed.

For now, we are going to add this code in the **MainWindow.xaml.cs** code-behind file. Later, in [Chapter 3](#), *MVVM for Maintainability and Testability*, this type of code will be added in a **ViewModel** file. The **MainWindow** will contain only presentation logic and will not be responsible for creating or fetching the data that populates the UI:

1. First, open **MainWindow.xaml.cs** and create a method named **PopulateData**. This method will contain the code that creates three **MediaItem** objects (a CD, a book, and a Blu-ray) and adds them to a private **List** named **_items**, as illustrated in the following code snippet:

```
public void PopulateData()
{
    if (_isLoading) return;
    _isLoading = true;
    var cd = new MediaItem
    {
        Id = 1,
        Name = "Classical Favorites",
        MediaType = ItemType.Music,
        MediumInfo = new Medium { Id = 1,
            MediaType = ItemType.Music, Name = "CD" }
    };
    var book = new MediaItem
    {
        Id = 2,
        Name = "Classic Fairy Tales",
        MediaType = ItemType.Book,
```

```

        MediumInfo = new Medium { Id = 2,
            MediaType = ItemType.Book, Name = "Book" }
    };
    var bluRay = new MediaItem
    {
        Id = 3,
        Name = "The Mummy",
        MediaType = ItemType.Video,
        MediumInfo = new Medium { Id = 3,
            MediaType = ItemType.Video, Name = "Blu Ray" }
    };
    _items = new List<MediaItem>
    {
        cd,
        book,
        bluRay
    };
}

```

You will need to add the **using** statements to **MainWindow** for the **MyMediaCollection.Model** and **MyMediaCollection.Enums** namespaces.

2. Define **_items** **IList** and **isLoading** **bool** as private class members. We will later change the collection of items to an **ObservableCollection**. **ObservableCollection** is a special collection that notifies data-bound items on the UI when items have been added or removed from the collection. For now, an **IList** will suit our needs. The code can be seen in the following snippet:

```

private IList<MediaItem> _items { get; set; }
private bool _isLoading;

```

3. Next, add a line of code to the **MainWindow** constructor, after the call to **InitializeComponent**, to call the new **PopulateData** method. Any code added to a **Window** or **UserControl** constructor must be added after this initialization code. This is where all the XAML code in **MainWindow.xaml** is initialized. If you attempt to reference any of

those elements before the call to **InitializeComponent**, it will result in an error. The constructor should now look like this:

```
public MainWindow()
{
    this.InitializeComponent();
    PopulateData();
}
```

We will return to this file to add some data-binding logic after creating some UI components in the XAML file. Let's go and do that now.

Building the initial UI

Earlier in the chapter, we added a single **TextBlock** to the **MainWindow.xaml** file with the **Text** property set to **Media**. We are going to add a **ListView** control beneath the existing **TextBlock**. A **ListView** control is a powerful and flexible control to display a list of items in a vertical list. It is like the **WinForms ListBox** control in basic list functionality (item selection, multi-selection, and automatic scroll bars), but each list item can be templated to display in just about any way imaginable.

NOTE

*For more information about the **ListView** class, documentation with sample code and markup is available on Microsoft Learn:*

<https://learn.microsoft.com/windows/windows-app-sdk/api/winrt/microsoft.ui.xaml.controls.listview>.

1. First, add some markup to create two rows inside the top-level **Grid** control. **RowDefinitions** should be added before the existing **TextBlock** element. **TextBlock** will remain in the first row, and we'll add the **ListView** control to the second row. Create **RowDefinitions** as illustrated in the following code snippet:

```
<Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*/>
</Grid.RowDefinitions>
```

As you can see, we're automatically sizing the first **RowDefinition** to the size of the **TextBlock** and allocating the remaining space inside the **Grid** to the **ListView** control's **RowDefinition** by using **"*"**. If you assign more than one row with **"*"**, those rows will split the remaining available space equally. This is the default value for the **Height** attribute. It would be sized the same if **"*"** were omitted, but most XAML developers explicitly include it for completeness and improved readability.

NOTE

For more information about row sizing, see the following Microsoft Learn documentation:

<https://learn.microsoft.com/windows/windows-app-sdk/api/winrt/microsoft.ui.xaml.controls.rowdefinition.height>.

2. Next, add a **ListView** control to the **Grid**, setting **Grid.Row** equal to **1** (row numbering is 0-based, meaning the first row is row **0**), and name it **ItemList**. It is not necessary to name your XAML elements. This is only required if you either want to reference them by **ElementName** in XAML data binding or as a variable in the code-behind file. Naming a control causes a variable to be created in the **InitializeComponent** call, and to optimize performance, you should only name your controls if you need to reference them elsewhere. The markup can be seen in the following snippet:

```
<ListView Grid.Row="1" x:Name="ItemList"
    Background="LightGoldenrodYellow"/>
```

By setting the background color to **LightGoldenrodYellow**, we can see the **ListView** in the application's main window before we populate any data. Depending on whether you are using a *Light* or *Dark* Windows theme, you can choose a color that works best for you. Run the app, and it should look something like this:

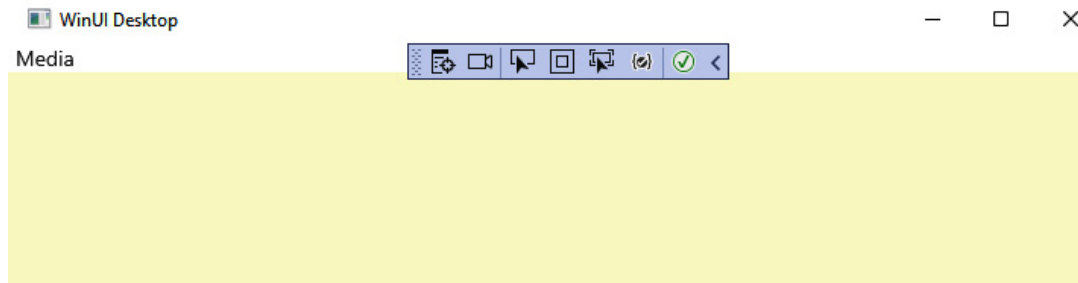


Figure 2.9 – A ListView added to the UI

Let's return to the code-behind file to start wiring up the data binding.

NOTE

If you are new to data binding concepts in XAML programming, Microsoft Learn has a great overview of data binding for Windows App SDK developers at the following web page:

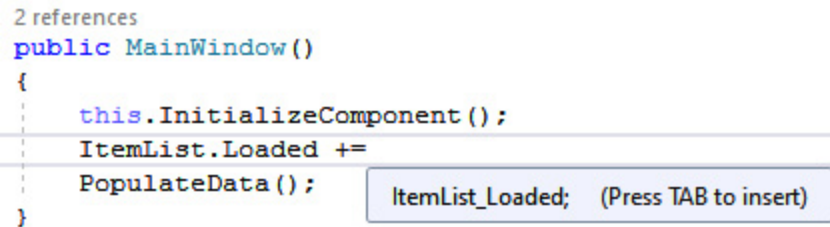
<https://learn.microsoft.com/windows/apps/develop/data-binding/>.

Completing the data-binding initialization

Return to the **MainWindow.xaml.cs** file and add a line of code before the **PopulateData()** method call, as follows:

```
ItemList.Loaded += ItemList_Loaded;
```

After typing **+=**, Visual Studio **IntelliSense** will prompt you to press *Tab* to automatically create the **ItemList_Loaded** event handler, as illustrated in the following screenshot:



The screenshot shows the `MainWindow` constructor in a C# file. The code is as follows:

```
2 references
public MainWindow()
{
    this.InitializeComponent();
    ItemList.Loaded +=
    PopulateData();
}
```

A tooltip is visible next to the `ItemList.Loaded +=` line, displaying the text: `ItemList_Loaded; (Press TAB to insert)`.

Figure 2.10 – Inserting the `ItemList_Loaded` event handler

Alternatively, if you type the entire line and press *Ctrl* +, you'll be prompted to create the new method.

To keep the `MainWindow` constructor simple and keep our data-loading code together, move the `PopulateData` call to the `ItemList_Loaded` method. Then, add the two other lines of code to the new event handler, as follows:

```
private void ItemList_Loaded(object sender,
    Microsoft.UI.Xaml.RoutedEventArgs e)
{
    var listView = (ListView)sender;
    PopulateData();
    listView.ItemsSource = _items;
}
```

This code is called after the `ItemsList` control has completed loading in the UI. We're getting the instance of the control from the `sender` parameter and setting the `ItemsSource` of the list to the `_items` collection that was loaded in `PopulateData()`. Now, we have data in the list, but things don't look quite right, as can be seen from the following screenshot:

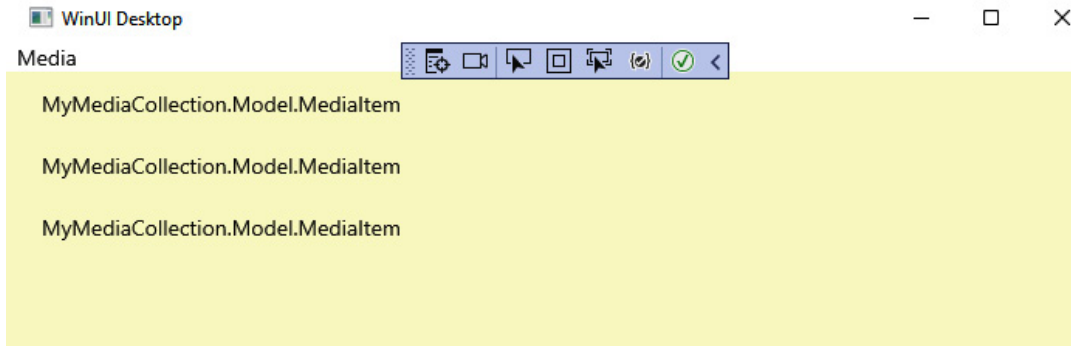


Figure 2.11 – A ListView with three rows of data

The **ListView** is displaying three rows for our three sample data items, but it's displaying the data type for each item instead of the data. That's because we haven't told the **ListView** which properties it should display for the items in the collection. By default, the list will display whatever is returned by an object's **ToString()** method. If **ToString()** is not overridden, the data type name of the class is returned.

Creating the DataTemplate and binding the UI

Let's return to **MainWindow.xaml** and tell the **ListView** which data we want to display for each item in the list. Try making this change with the application still running. Thanks to Visual Studio's **XAML Hot Reload** feature, you should see the UI reload without having to restart the debugging session.

Customizing the appearance of each **ListView** item is accomplished by defining a **DataTemplate** inside **ListView.ItemTemplate**. A **DataTemplate** can contain any WinUI controls we need to define the layout of each list item. Let's keep it simple and add a **Grid** containing two columns. Each column will contain a **TextBlock**. The **ListView** should now look like this:

```
<ListView Grid.Row="1"
```



```
x:Name="ItemList"
Background="LightGoldenrodYellow">
<ListView.ItemTemplate>
  <DataTemplate x:DataType="model:MediaItem">
    <Grid>
      <Grid.ColumnDefinitions>
        <ColumnDefinition Width="100"/>
        <ColumnDefinition Width="*/>
      </Grid.ColumnDefinitions>
      <TextBlock Text="{x:Bind
        Path=MediumInfo.Name}"/>
      <TextBlock Grid.Column="1"
        Text="{x:Bind Path=Name}"/>
    </Grid>
  </DataTemplate>
</ListView.ItemTemplate>
</ListView>
```

We are beginning to create some more complex XAML and need to keep formatting in mind to optimize its readability.

NOTE

*To quickly reformat your XAML, you can use the Ctrl + K | D keyboard shortcut. There is also an extension on the **Visual Studio Marketplace** called **XAML Styler**. You can search for it in Visual Studio's **Manage Extensions** dialog or get more information on the Marketplace here:*

<https://marketplace.visualstudio.com/items?itemName=TeamXavalon.XAMLStyler2022>.

To enable each **TextBlock** to bind to the properties of **MediaItem**, we must set an **x:DataType** property on the **DataTemplate**. To resolve **MediaItem**, a namespace declaration needs to be added to the **Window** definition, like this:

```
xmlns:model="using:MyMediaCollection.Model"
```

A shortcut to add this **using** statement is to place your cursor in **x:DataType** and press *Ctrl* +. Visual Studio will suggest adding the missing namespace to the file. We now have access to objects in the **MyMediaCollection.Model** namespace by using the **model** prefix, and **x:DataType="model:MediaItem"** will resolve **MediaItem** when we build and run the app.

Each **TextBlock** has its **Text** property bound to a property of **MediaItem**, using the **x:Bind** markup extension.

NOTE

*Using **x:Bind** instead of the **Binding** markup extension to bind data to the UI has the benefit of compile-time validation and increased performance. The previously noted data-binding overview on Microsoft Learn covers the differences in depth. I prefer to use **x:Bind** where possible. One important difference between **Binding** and **x:Bind** you should note is that while **Binding** defaults to **OneWay** mode, **x:Bind** defaults to **OneTime**. This change to the default binding behavior was made for performance considerations. **OneWay** binding requires more code behind the scenes to wire up the change detection needed for monitoring changes to the source value. You can still explicitly update your **x:Bind** usages to be **OneWay** or **TwoWay**. For more information about **x:Bind**, see this Microsoft Learn article:*

<https://learn.microsoft.com/windows/uwp/xaml-platform/x-bind-markup-extension>.

For more information on markup extensions in XAML, you can read this .NET article: <https://learn.microsoft.com/dotnet/desktop/xaml-services/markup-extensions-overview>.

Now, when you run the application, you can see the **MediumType** name and the item name for each item in the **ListView**, as illustrated in the following screenshot:

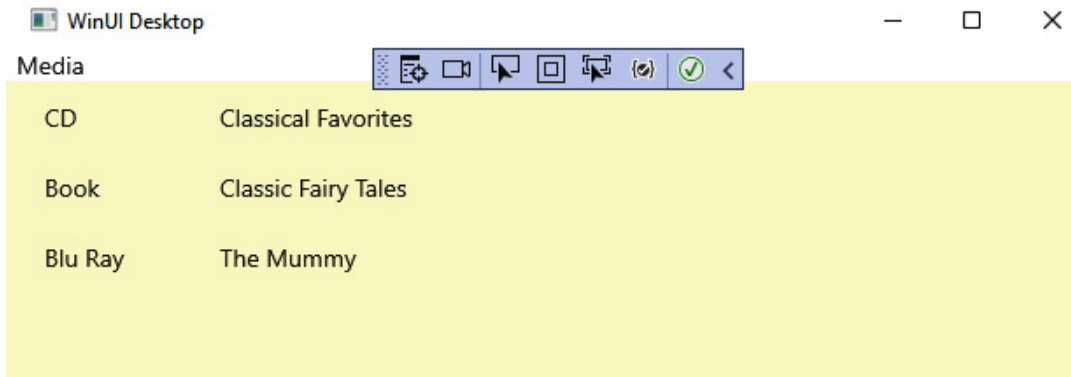


Figure 2.12 – The ListView with two columns of sample data

That's some pretty good progress! Before we expand on the functionality, let's talk a little about how WinUI, the Windows App SDK, and .NET fit together in terms of the app development process.

Understanding WinUI and Windows App SDK

Let's review the WinUI controls available to use in our project and see how they can help us build the **My Media Collection** application. The **Microsoft.WindowsAppSDK** package we saw in **Solution Explorer** earlier in the chapter contains these controls and much more.

To view the contents of this package, open the **Object Browser** window from Visual Studio's **View** menu. The controls will be listed here under several namespaces inside the **Microsoft.WinUI** tree node, as illustrated in the following screenshot:



Figure 2.13 – WinUI controls in Object Browser

The majority of WinUI controls we will be using can be found inside the **Microsoft.UI.Xaml.Controls** namespace under **Microsoft.WinUI**, along with other related classes and interfaces.

So far, we have used in our application the **Grid**, **TextBlock**, and **ListView** controls. Open the **Object Browser** window from the **View** menu. Then, find the **ListView** class, expand **Microsoft.UI.Xaml.Controls.ListView | Base Types**, and select the **ListViewBase** class. This base class contains the methods, properties, and events available to the **ListView**. The members of **ListViewBase**

will display in the right pane. Take some time to review these members and see whether you recognize any of them from your use of the control so far.

Clear **ListView** from your search, scroll down in the left pane of **Object Browser**, and find the **TextBlock** control. Select it and, in the right pane, find and select the **Text** property. The bottom-right pane displays details of the property, as illustrated in the following screenshot:

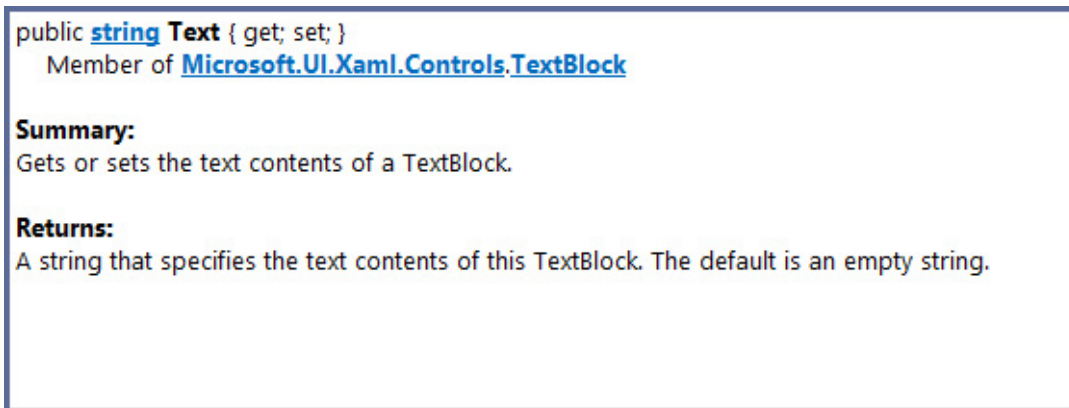


Figure 2.14 – Details of the TextBlock.Text property in Object Browser

The **Object Browser** window can be a valuable resource when familiarizing yourself with a new library or project. All referenced projects, NuGet packages, and other references will appear here.

The controls and other components you have reviewed here make up the UI layer of Windows App SDK applications. The underlying application framework for the Windows App SDK is .NET.

Understanding the .NET app model

You may have heard that the underlying app model for WinUI apps can be Win32 for C++ apps or .NET for desktop apps. So, what exactly is the **.NET app model**?

The .NET app model in WinUI apps describes how applications are packaged and deployed. It also defines the following behaviors and

capabilities:

- Data storage
- State management
- Lifecycle events (**Startup** and **Shutdown**)
- Multitasking
- Resource management
- Inter-app communication

WinUI 3 is one component of the Windows App SDK, the UI layer.

Although the WinUI controls are decoupled from the Windows SDK, the underlying .NET app platform is still dependent on it when using the Windows App SDK. Selecting a target and minimum Windows version is one of the side effects of this dependency.

Now that you have a better understanding of WinUI controls and how they relate to the Windows App SDK and the .NET app platform, let's use a few more of them in our application.

Working with WinUI controls, properties, and events

It's time to enhance the UI of the application. Currently, the main page only consists of a **Media** label over a **ListView**, with columns for the media type and the name of the media item. The following are the enhancements we will add in this section:

- A header row for the **ListView**
- A **ComboBox** to filter the rows based on the media type
- A **Button** to add a new item to the collection

We will start by enhancing the **ListView** for our media collection.

Adding a **ListView** header

Before we create the header, let's change the background color of the **ListView**. The **Aqua** color worked well to highlight the control, but it would be distracting when the application is used by our customers. We will discuss WinUI theme brushes and look at **Fluent Design** concepts later in [*Chapter 7, Fluent Design System for Windows Applications*](#). For now, just remove **Background="Aqua"** from the **ListView** definition in the **MainWindow.xaml** file.

Creating the header row for the media collection is relatively simple. To define the rows for each item, we created a **ListView.ItemTemplate** block containing a **DataTemplate**. To create the header, we do the same inside a **ListView.HeaderTemplate** block.

Just as with the item rows, the header row will consist of a **Grid** with two columns, with the same **Width** definitions. We again want to use two **TextBlock** controls inside the **Grid**, but to add some separation between the header and the items, we will add **Border** controls. Let's look at the markup for the header and then discuss the differences in more detail. Take some time to review the following markup:

```
<ListView.HeaderTemplate>
  <DataTemplate>
    <Grid>
      <Grid.ColumnDefinitions>
        <ColumnDefinition Width="100"/>
        <ColumnDefinition Width="*/>
      </Grid.ColumnDefinitions>
      <Border BorderBrush="BlueViolet"
        BorderThickness="0,0,0,1">
        <TextBlock Text="Medium"
          Margin="5,0,0,0"
          FontWeight="Bold"/>
      </Border>
      <Border Grid.Column="1"
        BorderBrush="BlueViolet"
        BorderThickness="0,0,0,1">
```



```

        <TextBlock Text="Title"
                Margin="5,0,0,0"
                FontWeight="Bold"/>
    </Border>
</Grid>
</DataTemplate>
</ListView.HeaderTemplate>

```

As you can see, each **TextBlock** is nested inside a **Border** element. This will wrap the text in a border with a **BlueViolet** color. However, by setting **BorderThickness="0,0,0,1"**, the border color will only appear on the bottom of the header row items. Here is how that appears in the application:

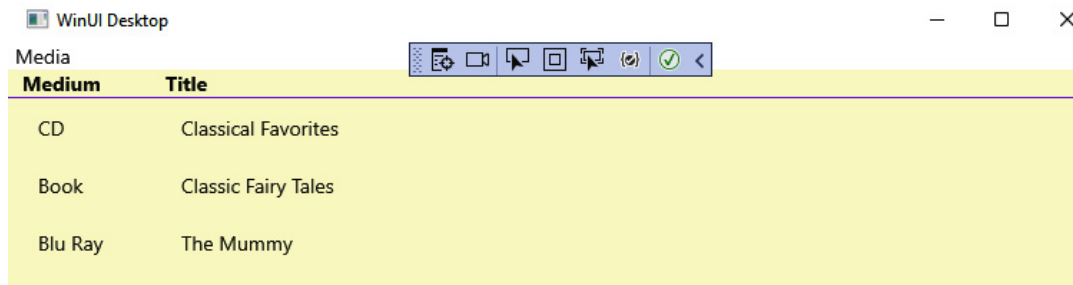


Figure 2.15 – The ListView with a header row added

NOTE

*The same bottom border could be achieved by nesting the entire **Grid** inside a **Border** instead of putting one around each header item. However, by doing it this way, we have more control over the appearance of each column's border style. When we implement sorting later, the border's color can be modified to highlight the column on which sorting has been applied.*

You probably also noticed that the header row text stands out from the rows in the grid. The **FontWeight="Bold"** property set inside each **TextBlock** in the **HeaderTemplate** helps to highlight the header row.

Creating the ComboBox filter

One of the requirements for the application is to allow users to filter several of the collection items' properties. Let's start simple, by adding a filter only on the medium (**Book**, **Music**, or **Movie**). The list also needs an **All** option, which will be the default selection when users open the application:

1. First, add some XAML to **MainWindow** to add a filter to the right of the **Media** label. Replace **Media TextBlock** with the following markup:

```
<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*/>
        <ColumnDefinition Width="Auto"/>
    </Grid.ColumnDefinitions>
    <TextBlock Text="Media Collection"
        Margin="4"
        FontWeight="Bold"
        VerticalAlignment="Center"/>
    <StackPanel Grid.Column="1"
        Orientation="Horizontal"
        HorizontalAlignment="Right">
        <TextBlock Text="Media Type:" Margin="4"
            FontWeight="Bold"
            VerticalAlignment="Center"/>
        <ComboBox x:Name="ItemFilter"
            MinWidth="120" Margin="0,2,6,4"/>
    </StackPanel>
</Grid>
```

The single **TextBlock** label has been replaced with a two-column **Grid**. The first column contains the **TextBlock**, with a few modifications. First, the **Text** property has been updated to "**Media Collection**". **FontWeight** has been changed to "**Bold**" and some margin has been added. Finally, the element is vertically centered. The second column contains a new **StackPanel** (a container control that stacks its contents horizontally or vertically). The default ori-

entation is **Vertical**. In our case, we want a horizontal stack, which is why the **Orientation** property has been set.

StackPanel contains a **TextBlock** label and a **ComboBox** for the filter selection. The **ComboBox** filter has been given an **x:Name** so we can reference it from the C# code-behind file when initializing its contents. We have also configured a **MinWidth** value of **120**. If the contents of the **ComboBox** filter require more than 120 **pixels (px)**, it can grow larger, but its width cannot be less than the value set here.

NOTE

The pixels being referenced in XAML are effective pixels. To read more about responsive layouts and sizing with XAML, see this Microsoft Learn article:

<https://learn.microsoft.com/windows/apps/design/layout/layouts-with-xaml>.

1. In the **MainWindow.xaml.cs** file, add a new variable to hold the list of mediums, as follows:

```
private IList<string> _mediums { get; set; }
```

This collection can be an **IList** rather than **ObservableCollection** because we don't expect its contents to change while the application is running.

2. Inside the **PopulateData()** method, add some code at the end of the method to populate the **_mediums** list, as follows:

```
_mediums = new List<string>  
{  
    "All",  
    nameof(ItemType.Book),  
    nameof(ItemType.Music),
```

```
nameof(ItemType.Video)  
};
```

We're adding an item to the collection for each of the possible values in our **ItemType** enum, plus the default "All" value.

3. The **ComboBox** filter will be bound to the collection after it has loaded, so add a **Loaded** event handler in the **MainWindow** constructor, as we did for the **ItemList** earlier, like this:

```
ItemFilter.Loaded += ItemFilter_Loaded;
```

4. The **ItemFilter_Loaded** event handler will look much like the **ItemList_Loaded** handler. Use the following code:

```
private void ItemFilter_Loaded(object sender,  
    Microsoft.UI.Xaml.RoutedEventArgs e)  
{  
    var filterCombo = (ComboBox)sender;  
    PopulateData();  
    filterCombo.ItemsSource = _mediums;  
    filterCombo.SelectedIndex = 0;  
}
```

The code casts the sender to the **ComboBox** data type and sets its **ItemSource** to the list we populated in the previous step. Finally, an additional step is needed to default the **ComboBox** filter to default to the "All" item. This is accomplished by setting **SelectedIndex** equal to 0.

Let's run the application and see how it looks now. You can see the result in the following screenshot:

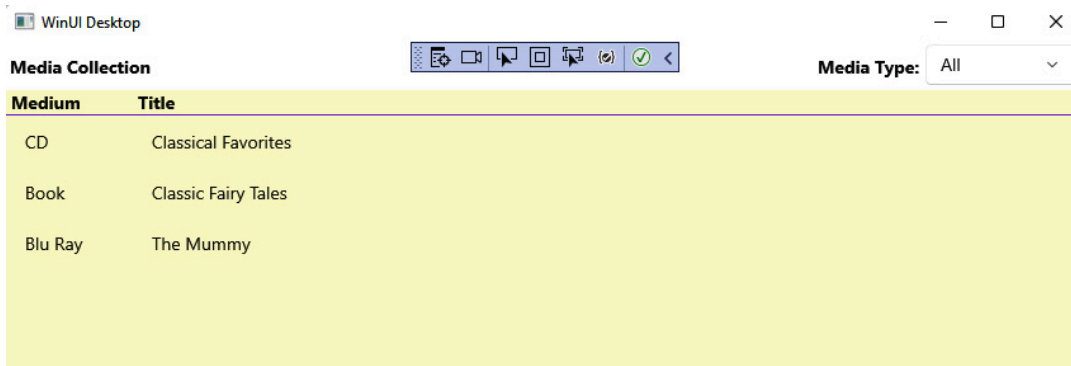


Figure 2.16 – Media Collection with the Media Type filter added

Pretty sharp! If you click the **Media Type** filter, you can see the four values available for selection, as illustrated here:

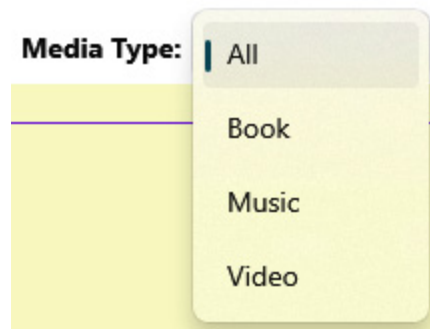


Figure 2.17 – Media Type values

Notice how the dropdown of **ComboBox** has picked up the Windows style with some transparency without having to add any additional code or markup. Select one of the other values in the list and see what happens. Nothing! That's because we haven't added any code to do the filtering when the selection changes on the filter. We can add a little bit of extra code to fix that, as follows:

1. First, create a new **_allItems** collection to store a list of all available media items, regardless of the current filter, like this:

```
private IList<MediaItem> _allItems { get; set; }
```

2. Next, in the **PopulateData()** method, after populating the **_items** collection, add the same items to **_allItems**, like this:

```
_allItems = new List<MediaItem>
{
    cd,
    book,
    bluRay
};
```

3. Now, we need to do some filtering when the filter selection changes. We want to handle the **SelectionChanged** event on the **ComboBox** control, but we don't want to hook it up until after the entire page has loaded. This will prevent the event from being handled while the **ComboBox** filter is initially being populated.
4. At the end of the implementation of **ItemFilter_Loaded**, add an event handler for the **SelectionChanged** event on the **ComboBox** control:

```
ItemFilter.SelectionChanged +=
    ItemFilter_SelectionChanged;
```

5. In the new **ItemFilter_SelectionChanged** event handler, we will iterate through the **_allItems** list and determine which of the items to include in the filtered list, based on their **MediaType** property, as follows:

```
private void ItemFilter_SelectionChanged(
    object sender, Microsoft.UI.Xaml.Controls.
        SelectionChangedEventArgs e)
{
    var updatedItems =
        (from item in _allItems
         where
             string.IsNullOrEmpty(ItemFilter.
                 SelectedValue.ToString()) ||
             ItemFilter.SelectedValue.ToString() ==
                 "All" ||
             ItemFilter.SelectedValue.ToString() ==
                 item.MediaType.ToString())
```

```
        select item).ToList();  
        ItemList.ItemsSource = updatedItems;  
    }
```

If the filter value is empty or **All** is selected, we want to include the item, regardless of its **MediaType**. Otherwise, we check whether the **MediaType** matches the selection in **ItemFilter ComboBox**. When there is a match, we add it to the **updatedItems** list. Then, we set **update-dItems** as **ItemsSource** on the **ListView**.

NOTE

A filter should never be empty unless there is an error while initializing the data. This condition is only a safeguard for unforeseen scenarios.

Now, run the app again and select **Book** in the filter, as illustrated in the following screenshot:

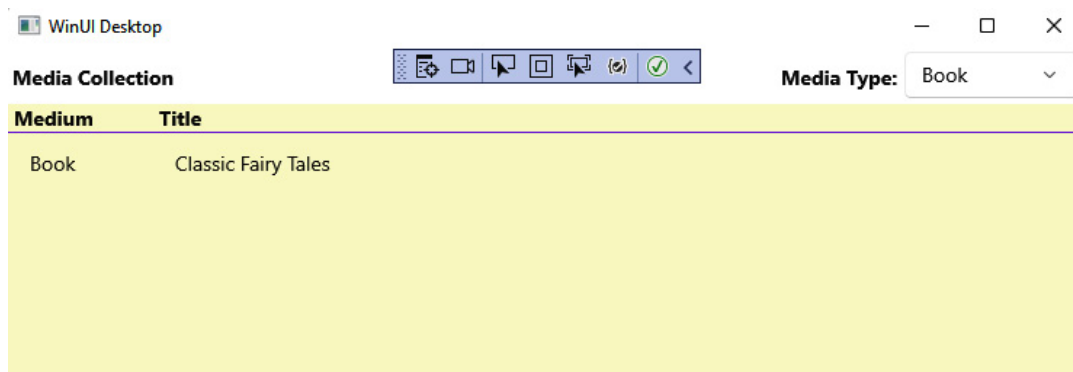


Figure 2.18 – Media Collection filtered to display only books

That takes care of the filter implementation for the time being. Let's finish up this part of the UI design with a **Button**.

Adding a new item button

We are not quite ready to start working with multiple windows or navigation yet. You should have some understanding of the **MVVM** pattern before we put too much logic into the app. This will minimize

our code refactoring in later chapters. However, we can add a **Button** to the current page and add some code to ensure everything is hooked up correctly.

Open the **MainWindow.xaml** file and add a third **RowDefinition** to the top-level **Grid** on the **Window**, as follows:

```
<Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*/>
    <RowDefinition Height="Auto"/>
</Grid.RowDefinitions>
```

The new row will have a height of **Auto** so that it sizes itself to fit the **Button**. We still want the **ListView** to take up most of the screen.

Now, after the closing tag of the **ListView** control, add the new **Button**, as follows:

```
<Button x:Name="AddButton" Content="Add Item"
    HorizontalAlignment="Right" Grid.Row="2" Margin="8"/>
```

As discussed in the previous chapter, the **Button** control does not have a **Text** property. Instead, if you only want a **Button** control to contain text, you assign it to the **Content** property. We are also assigning the **Button** control to the third row of the **Grid**, setting the margin, and aligning it to the right side of the **Grid**.

Let's see how the app looks now, as follows:

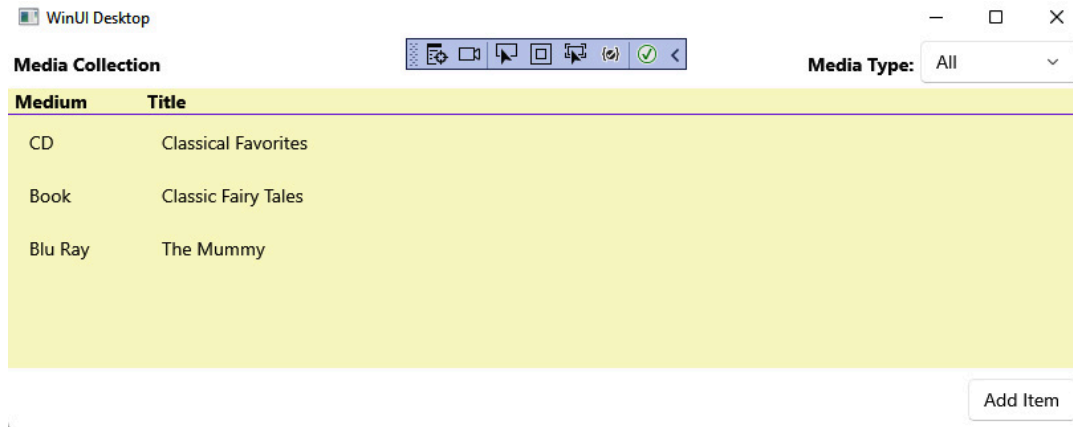


Figure 2.19 – My Media Collection with an Add Item button

The button doesn't do anything yet. Because we're not yet ready to add an additional window to the application to add items, let's open a message popup to inform the user that this function is not available.

In **MainWindow.xaml**, wire up a new event handler inside **AddButton**. You can also remove the **x:Name** attribute. We're able to remove the name because we do not need to reference it in the code-behind file. The code is shown in the following snippet:

```
<Button Content="Add Item"
        HorizontalAlignment="Right"
        Grid.Row="2" Margin="8" Click="AddButton_Click"/>
```

You can create the event handler by placing the cursor on the name of the handler, **AddButton_Click**, and pressing *F12*. This will create the handler and navigate to it in the **MainWindow.xaml.cs** file. Inside the **AddButton_Click** event handler, we will create a new **ContentDialog** with the message we want to display to the user.

NOTE

*The call to **dialog.ShowAsync()** must be awaited, so remember to add the **async** directive to the event handler, as shown next.*

The **async** directive is highlighted in the following code snippet for the new event handler:

```
private async void AddButton_Click(object sender,
    RoutedEventArgs e)
{
    var dialog = new ContentDialog
    {
        Title = "My Media Collection",
        Content = "Adding items to the collection is
            not yet supported.",
        CloseButtonText = "OK",
        XamlRoot = Content.XamlRoot
    };
    await dialog.ShowAsync();
}
```

Now, run the application again and click the **Add Item** button, as illustrated in the following screenshot:

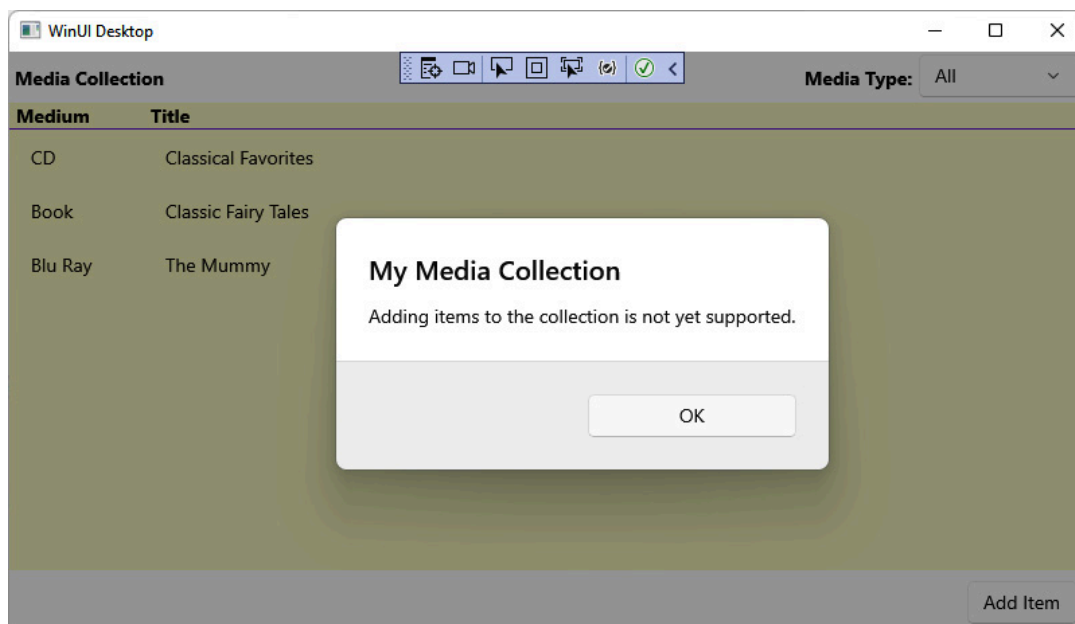


Figure 2.20 – Displaying a ContentDialog popup

That's all there is to adding a functional **Button** to a WinUI page. As we have discussed, some of this code will change and be moved to a

ViewModel in the next chapter, but you should now have a good idea of how to work with some basic **Button** properties and events.

Summary

We have made some great initial progress on the **My Media Collection** application in this chapter. Along the way, you have learned how to use several common WinUI controls. You have also learned how to change the appearance, layout, and behavior of WinUI controls by using different layout controls and updating control properties in XAML. Finally, you saw how to leverage data binding and events to add and update data displayed to the user.

Next, we will learn how to decouple some of the logic we have been writing in the code-behind files to build testable and maintainable applications.

Questions

1. How do you add or remove features to or from Visual Studio?
2. What is the lowest minimum version of Windows that must be targeted when creating a new WinUI 3 project?
3. Where can you add XAML resources that can be shared by components in a whole application?
4. What is the default name of the first window loaded in a new WinUI app?
5. Which XAML container control allows you to define rows and columns to lay out its contents?
6. Which XAML container control stacks its contents horizontally or vertically?
7. What is the name of the message box that WinUI apps can use to display simple messages to users?

8. Challenge: What type of layout panel in WinUI allows its contents to be positioned absolutely?